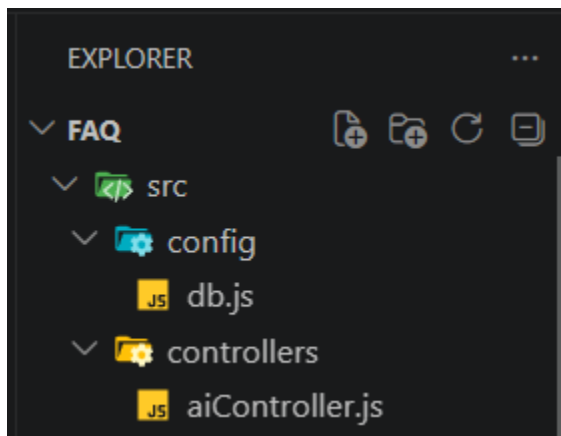


AI FAQ ASSISTANT API

5. Project Development Phase

5.1 Creating Project Folder

1. Create a new folder with your <project name>.
2. Inside that folder, create a new subfolder named src.
3. Now open that folder in VS Code.



Server Setup (npm init)

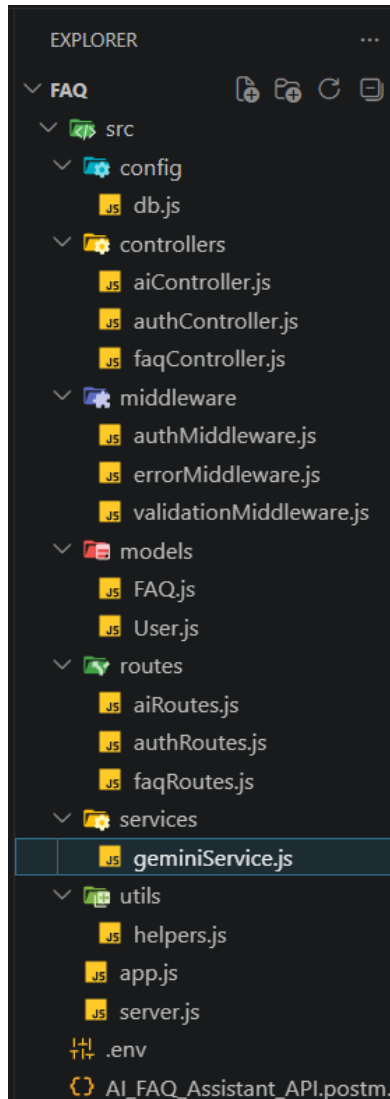
Open your root folder terminal in VS Code and run:

Bash

```
JavaScript
npm init -y
npm install express mongoose dotenv cors jsonwebtoken
bcrypt @google/genai
```

File and Folder Structure Matrix

Plaintext



5.2 Backend Development

Core System Script Implementations

1. Gateway Server Node (src/server.js)

This main entrance mechanism aggregates environment variables, triggers database listeners, and binds network listeners to active environment ports:

```

c> serverjs > ...
1 const app = require('./app');
2 const connectDB = async () => {
3   try {
4     const dbConnect = require('./config/db');
5     await dbConnect();
6   } catch (error) {
7     console.error('Failed to connect to the database', error);
8   }
9 };
10
11 const startServer = async () => {
12   // Connect to database
13   await connectDB();
14
15   const PORT = process.env.PORT || 5000;
16
17   const server = app.listen(PORT, () => {
18     console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
19   });
20
21   // Handle unhandled promise rejections
22   process.on('unhandledRejection', (err, promise) => {
23     console.error('Unhandled Rejection: ${err.message}');
24     // Close server & exit process
25     server.close(() => process.exit(1));
26   });
27 };
28
29 startServer();
30

```

2. Gateway Express Setup (src/app.js)

```

src> appjs > ...
1 const express = require('express');
2 const cors = require('cors');
3 const dotenv = require('dotenv');
4 const { errorHandler } = require('./middleware/errorMiddleware');
5
6 // Load environment variables
7 dotenv.config();
8
9 const app = express();
10
11 // Enable CORS
12 app.use(cors());
13
14 // Body parser middlewares
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: false }));
17
18 // Basic status check route
19 app.get('/', (req, res) => {
20   res.json({
21     success: true,
22     message: 'AI FAQ Assistant API is running.',
23     version: '1.0.0',
24   });
25 });
26
27 // Register API Routes
28 app.use('/api/auth', require('./routes/authRoutes'));
29 app.use('/api/faqs', require('./routes/faqRoutes'));
30 app.use('/api/ai', require('./routes/aiRoutes'));
31
32 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
33 app.use('/socket.io', (req, res) => {
34   res.status(404).json({ error: 'Socket.IO not enabled' });
35 });
36

```

3. Security Access Middleware (src/middleware/authMiddleware.js)

Intercepts incoming target queries, extracts transmission header authorization records, and validates token identity context claims:

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:

```

src > models > User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcrypt');
3
4  const userSchema = new mongoose.Schema(
5  {
6    name: {
7      type: String,
8      required: [true, 'Name is required'],
9      trim: true,
10   },
11   email: {
12     type: String,
13     required: [true, 'Email is required'],
14     unique: true,
15     lowercase: true,
16     trim: true,
17     match: [/^([a-zA-Z0-9_\.\-])+\@([a-zA-Z0-9_\.\-])+\./, 'Please provide a valid email address'],
18   },
19   password: {
20     type: String,
21     required: [true, 'Password is required'],
22     minlength: [6, 'Password must be at least 6 characters long'],
23   },
24 },
25 {
26   timestamps: true,
27 }
28 );
29
30 // Pre-save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });

```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:

```

src > models > FAQ.js > ...
1  const mongoose = require('mongoose');
2
3  const faqSchema = new mongoose.Schema(
4  {
5    question: {
6      type: String,
7      required: [true, 'Question is required'],
8      trim: true,
9    },
10   answer: {
11     type: String,
12     required: [true, 'Answer is required'],
13     trim: true,
14   },
15   category: {
16     type: String,
17     required: [true, 'Category is required'],
18     enum: {
19       values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20       message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking,
21     },
22     default: 'General',
23   },
24   createdBy: {
25     type: mongoose.Schema.Types.ObjectId,
26     ref: 'User',
27     required: [true, 'Creator user reference is required'],
28   },
29 },
30 {
31   timestamps: true,
32 }
33 );
34
35 const FAQ = mongoose.model('FAQ', faqSchema);
36

```

API Router Interface Endpoints

1. Public Authentication Operations (src/routes/authRoutes.js)

```
.env  geminiService.js  authRoutes.js  AI_FAQ_Assistant_API.postman_collection.json  B...
src > routes > authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    registerUser,
5    loginUser,
6    getUserProfile,
7  } = require('../controllers/authController');
8  const { protect } = require('../middleware/authMiddleware');
9  const {
10    validateRegister,
11    validateLogin,
12  } = require('../middleware/validationMiddleware');
13
14  // Public routes
15  router.post('/register', validateRegister, registerUser);
16  router.post('/login', validateLogin, loginUser);
17
18  // Protected routes
19  router.get('/profile', protect, getUserProfile);
20
21  module.exports = router;
22
```

2. Protected FAQ Operations (src/routes/faqRoutes.js)

JavaScript

```
.env  geminiService.js  faqRoutes.js  AI_FAQ_Assistant_API.postman_collection.json
> routes > faqRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    createFAQ,
5    getAllFAQs,
6    getFAQById,
7    updateFAQ,
8    deleteFAQ,
9    searchFAQs,
10  } = require('../controllers/faqController');
11  const { protect } = require('../middleware/authMiddleware');
12  const { validateFAQ } = require('../middleware/validationMiddleware');
13
14  // Public routes - GET operations
15  router.get('/search', searchFAQs); // Note: defined before /:id to prevent routing issues
16  router.get('/', getAllFAQs);
17  router.get('/:id', getFAQById);
18
19  // Protected routes - Modification operations
20  router.post('/', protect, validateFAQ, createFAQ);
21  router.put('/:id', protect, updateFAQ);
22  router.delete('/:id', protect, deleteFAQ);
23
24  module.exports = router;
25
```

3. AI Automation Operations (src/routes/aiRoutes.js)

```
rc > routes > aiRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    generateAIAnswer,
5    generateAIFAQ,
6  } = require('../controllers/aiController');
7  const { protect } = require('../middleware/authMiddleware');
8  const {
9    validateAIAnswer,
10   validateAIFAQ,
11 } = require('../middleware/validationMiddleware');
12
13 // Protected routes to prevent API overuse/abuse
14 router.post('/answer', protect, validateAIAnswer, generateAIAnswer);
15 router.post('/generate-faq', protect, validateAIFAQ, generateAIFAQ);
16
17 module.exports = router;
18
```